

神经网络基础

学习目标

Logistic回归

Logistic回归

逻辑回归的损失函数

梯度下降算法

导数

导数

导数计算图

链式法则

逻辑回归的梯度下降

向量化编程

向量化优势

向量化实现伪代码

正向传播与方向传播

作业实现：

学习目标

- 目标
 - 知道逻辑回归的算法计算输出、损失函数
 - 知道导数的计算图
 - 知道逻辑回归的梯度下降算法
 - 知道多样本的向量计算
- 应用
 - 应用完成向量化运算
 - 应用完成一个单神经元神经网络的结构

Logistic回归

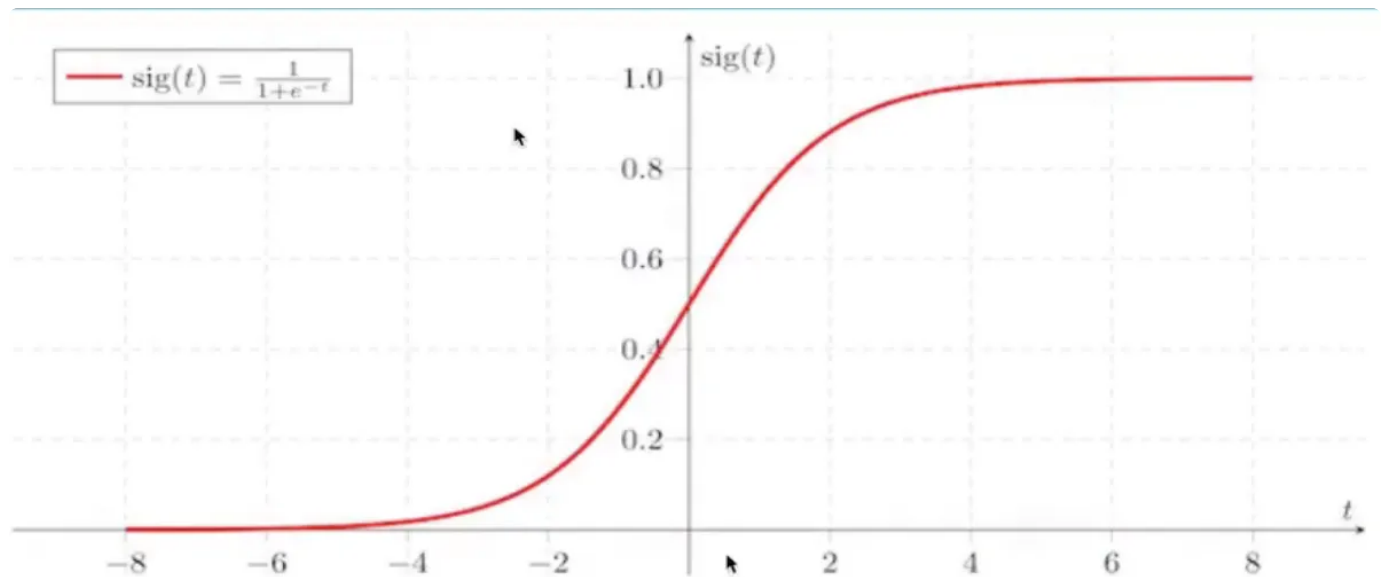
Logistic回归

逻辑回归是一个主要用于二分类的算法，那么逻辑回归是给定一个 x ，输出一个该样本属于1对应类别的预测概率 $\hat{y} = P(y = 1|x)$ 。

Logistic回归中使用的参数如下：

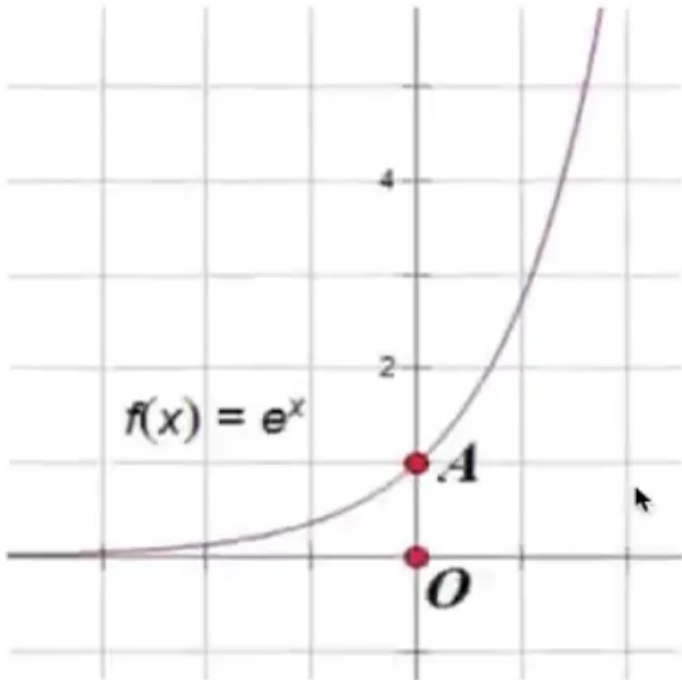
- 输入的特征向量： $x \in R^{n_x}$, x 是一个 n_x 维的特征数量，用于训练的标签： $y \in 0, 1$
- 参数：权重： $w \in R^{n_x}$, 偏置： $b \in R$
- 输出预测结果： $\hat{y} = \sigma(w^T x + b) = \sigma(w_1 x_1 + w_2 x_2 + \dots + b) = \sigma(\theta^T x)$
 - Sigmoid 函数： $s = \sigma(w^T x + b) = \sigma(z) = \frac{1}{1+e^{-z}}$
 - 如果 z 的结果非常大，那么S的结果接近于1
 - 如果 z 的结果较小或者是非常大的负数，那么S的结果接近于0

sigmoid函数图像：



1. z 非常大，约等于 $1/1+0=1$
2. z 非常的小或者负数，约等于 $1/1+\text{无穷}=0$

e^{-z} 的函数如下



例如：

例如：

样本特征值输入			回归	逻辑回归结果		真实结果
W ×	12.3	20.0	16	82.4	0.4	1
	9.4	21.1	7.2	89.1	sigmoid 0.68	0
	34.4	18.7	8.1	80.2	→ 0.41	1
	10.2	16.0	12.5	81.3	→ 0.55	0
	5.6	10.0	6.3	90.4	→ 0.71	1

主要公式为： $\hat{y} = \sigma(w^T x + b) = \sigma(w_1 x_1 + w_2 x_2 + \dots + b) = \sigma(\theta^T x)$

输出P (y=1|x)

$(1,n)$	$(n,1)$
$w = (w_1$	$x = (x_1$
w_2	x_2
w_3	x_3
$\dots$	$\dots$
$w_n)$	$x_n)$

逻辑回归的损失函数

损失函数用于衡量预测结果与真实值之间的误差。最简单的损失函数定义方式为平方差损失：

$$L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2 \quad \text{因为又多个局部最小点所以才有对应的损失函数}$$

逻辑回归一般使用: $L(\hat{y}, y) = -(y \log \hat{y}) - (1 - y) \log(1 - \hat{y})$

该式子的理解:

- 如果 $y=1$,损失为 $-\log \hat{y}$,那么要想损失越小, $\hat{y}$ 的值必须越大,即越趋近于或者等于1
- 如果 $y=0$,损失为 $-\log(1 - \hat{y})$,那么要想损失越小,那么 $\hat{y}$ 的值越小,即趋近于或者等于0

损失函数是在单个训练样本中定义的,它衡量了在单个训练样本上的表现。代价函数(cost function)衡量的是在全体训练样本上的表现,即衡量参数 w 和 b 的效果,所有训练样本的损失平均值

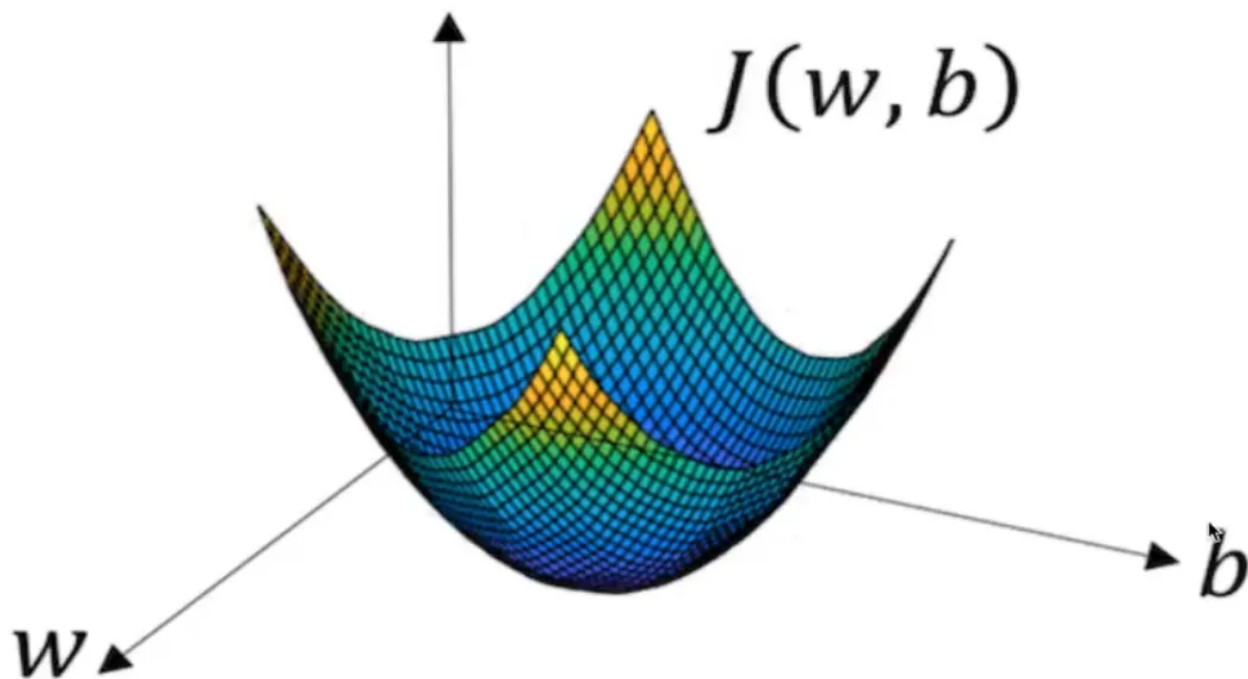
$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

梯度下降算法

目的: 使损失函数的值找到最小值

方式: 梯度下降

函数的而梯度指出了函数最陡增长方向。梯度的方向走,函数增长的得就越快。那么按梯度得负方向走,函数值自然就降低得最快了。模型得训练目标即是寻找合适得 w 与 b 以最小化代价函数得值。假设 w 和 b 都是一维实数,那么可以得到如下得 J 关于 w 和 b 的图:



最低点为我们要找的 w 和 b 值,寻找的过程是一个迭代的过程。

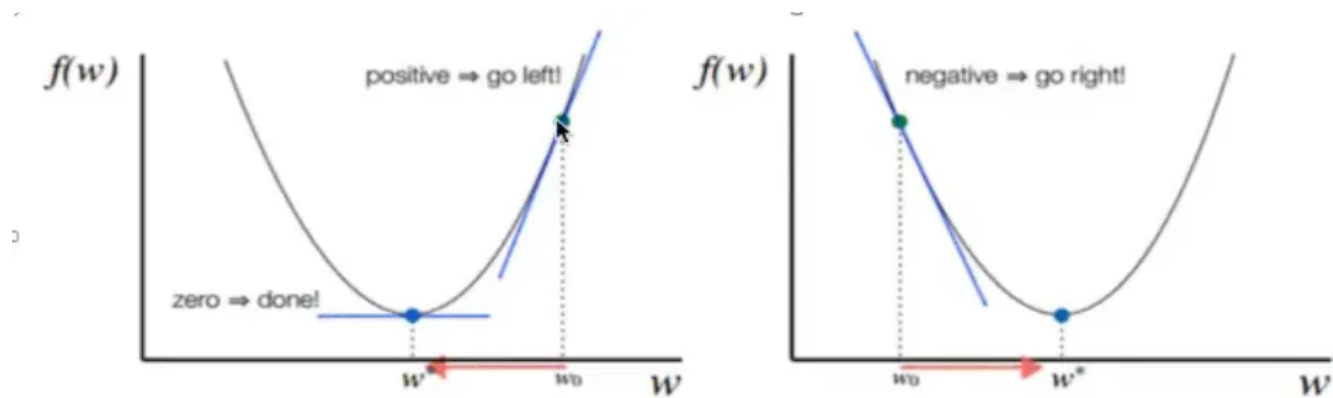
成本 J 函数是一个凸函数,与非凸函数的区别在于其不含多个局部最低。

参数 w 和 b 的更新公式为: dw 为 w 的导数

$$w_k := w - \alpha \frac{dJ(w, b)}{dw}, \quad b := b - \alpha \frac{dJ(w, b)}{db}$$

注：其中 α 表示学习速率，即每次更新的 w 的步伐长度。当 w 大于最优解 w' 时，导数大于 0，那么 w 就会向更小的方向更新。反之当 w 小于最优解 w' 时，导数小于 0，那么 w 就会向更大的方向更新。迭代直到收敛。

通过平面来理解梯度下降过程：



导数

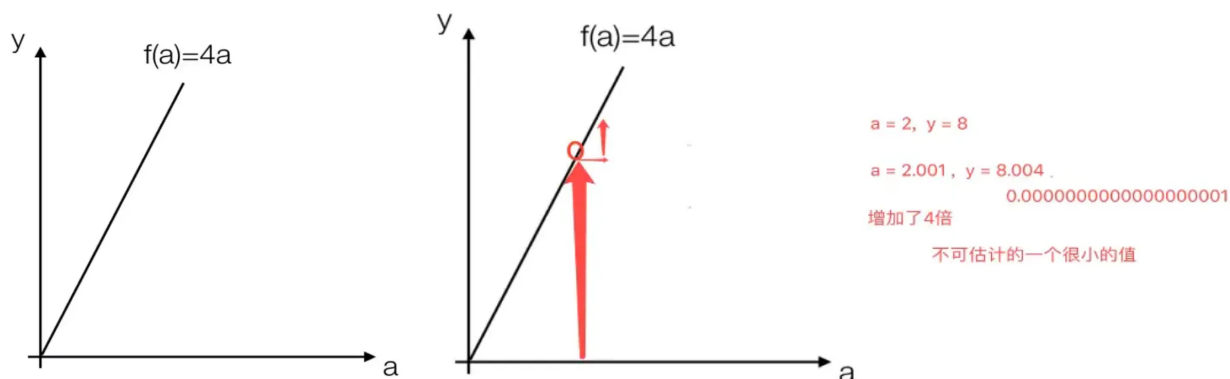
理解梯度下降的过程之后，我们通过例子来说明梯度下降在计算导数意义或者说这个导数的意义。

导数

导数也可以理解成某一点处的斜率。斜率这个词更直观一些。

斜率：当增加自变量一定不可估计的小的值，应变量增加的相对于自变量的倍数

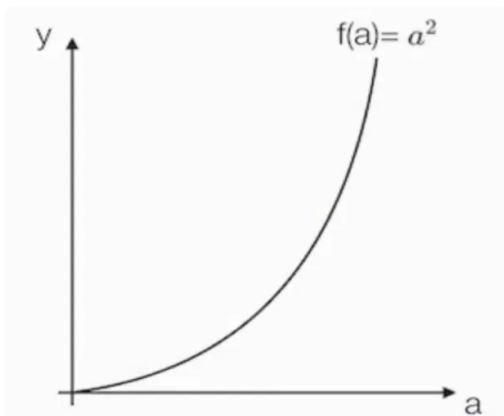
- 各点处的导数值一样



那么我们的这个斜率可以理解为当一个点偏移一个不可估量的小的值，所增加的为4倍。

可以记作 $\frac{f(a)}{da}$ 或者 $\frac{d}{da} f(a)$

- 各点的导数值不全一致



例：取一点为 $a=2$,那么 y 的值为4，我们稍微增加 a 的值为 $a=2.001$,那么 y 的值约等于4.004（4.004001），就是当 a 增加了0.001，随后 y 增加了4倍

取一点为 $a=5$,那么 y 的值为25，我们稍微增加 a 的值为 $a=5.001$,那么 y 的值约等于25.01（25.010001），也是当 a 增加了0.001，随后 y 增加了10倍

对于 a^2 ，这个函数，所有点的导数都可以表示为 $2a$

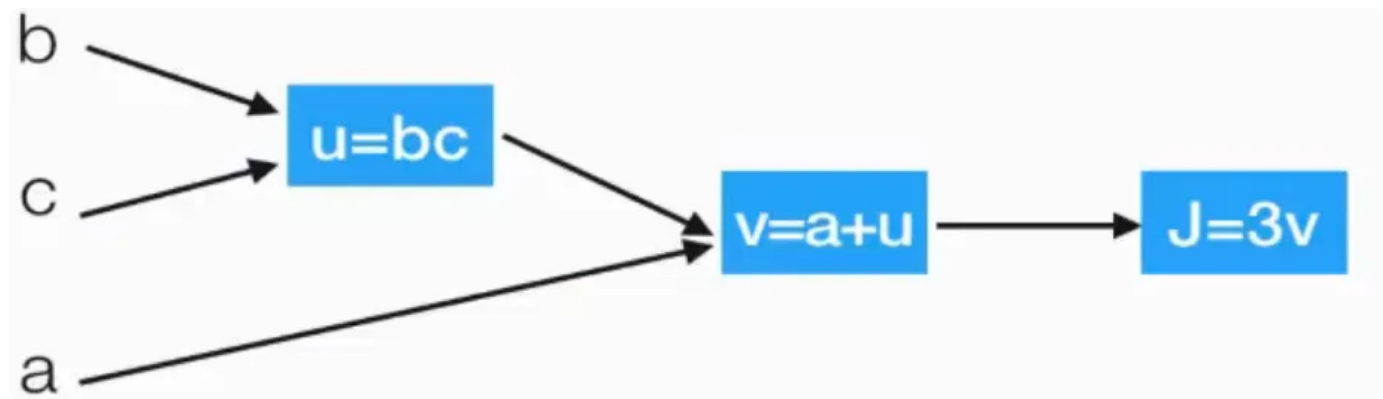
- 更多函数的导数结果

函数	导数
$f(a) = a^2$	$2a$
$f(a) = a^3$	$3a^2$
$f(a) = \ln(a)$	$\frac{1}{a}$
$f(a) = e^a$	e^a
$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(z)(1 - \sigma(z))$
$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - (\tanh(z))^2 = 1 - (g(z))^2$

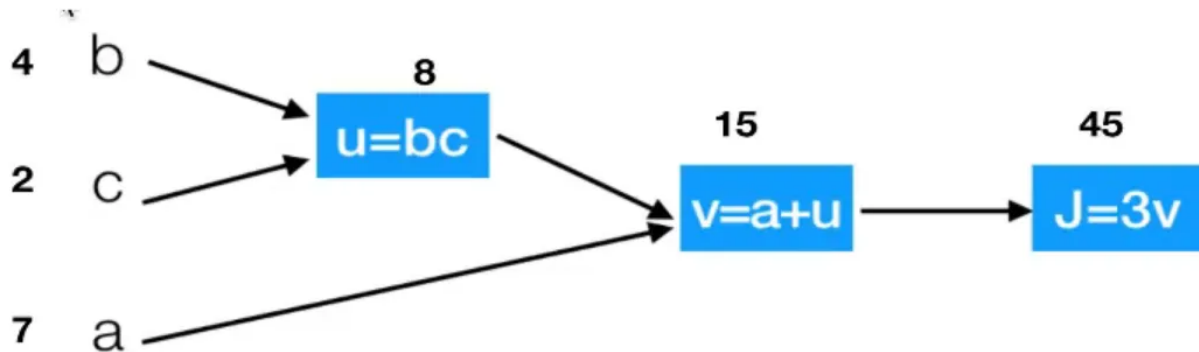
导数计算图

那么接下来我们来看看含有多个变量的到导数流程图，假设 $J(a, b, c) = 3(a + bc)$

我们以下面流程图代替：



这样就相当于从左到右计算出结果，然后从后往前计算出导数过程



$$dv = \frac{dj}{dv} = 3 \quad \mathbf{V \ 15.001, J \ 45.003}$$

$$da = \frac{dj}{da} = 3 \quad \mathbf{a=7 \rightarrow a=7.001, \ v = 15.001, \ j \rightarrow 45.003}$$

$$da = \frac{dj}{da} = \frac{dj}{dv} * \frac{dv}{da} = 3 * 1 = 3$$

- 导数计算

问题：那么现在我们要计算j相对于三个变量a, b, c的导数？

假设b=4, c=2、a=7, u=8、v=15、j=45

- $\frac{dJ}{dv} = 3$

增加v从15到15.0001, 那么J约等于45.003

- $\frac{dJ}{da} = 3$

增加a从7到7.0001, 那么v≈15.001, J≈45.003

这里涉及到链式法则

链式法则

- $\frac{dJ}{da} = \frac{dJ}{dv} \frac{dv}{da} = 3 * 1 = 3$

J相对于a增加的量可以理解为J相对于v*v相对于a增加的

接下来计算

- $\frac{dJ}{db} = 6 = \frac{dJ}{du} \frac{du}{db} = 3 * 2$
- $\frac{dJ}{dc} = 12 = \frac{dJ}{du} \frac{du}{dc} = 3 * 4$

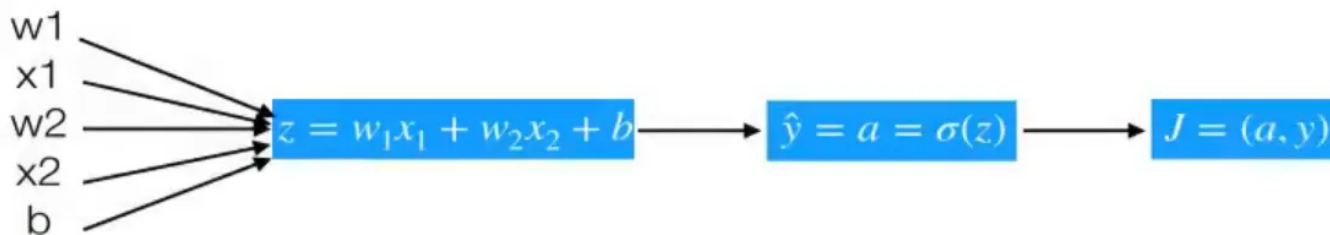
$$db = \frac{dj}{db} = \frac{dj}{du} * \frac{du}{db} = 3 * 2 = 6 \quad \mathbf{u \rightarrow 8.001, v \rightarrow 15.001, J \rightarrow 45.003}$$
$$dc = \frac{dj}{dc} = \frac{dj}{du} * \frac{du}{dc} = 3 * 4 = 12 \quad \mathbf{b \rightarrow 4.001, u \rightarrow 8.002}$$
$$\quad \quad \quad \mathbf{c \rightarrow 2.001, u \rightarrow 8.004}$$

逻辑回归的梯度下降

逻辑回归的梯度下降过程计算图，首先从前往后的计算图得出如下

- $z = w^T x + b$
- $\hat{y} = a = \sigma(z)$
- $L(\hat{y}, y) = -(y \log a) - (1 - y) \log(1 - a)$

那么计算图从前向过程为，假设样本有两个特征



问题：计算出J关于z的导数

$$dz = \frac{dJ}{da} \frac{da}{dz} = a - y$$

- $$\frac{dJ}{da} = -\frac{y}{a} + \frac{1-y}{1-a}$$

$$\frac{da}{dz} = a(1-a)$$

所以我们这样可以求出总损失相对于w1, w2, b参数的某一点导数，从而可以更新参数

$$\frac{dJ}{dw_1} = \frac{dJ}{dz} \frac{dz}{dw_1} = dz * x_1$$

- $$\frac{dJ}{dw_2} = \frac{dJ}{dz} \frac{dz}{dw_2} = dz * x_2$$

$$\frac{dJ}{db} = dz$$

我们计算 损失函数的某个点相对于w1, w2, b的导数之后，就可以更新这次优化后的结果

$$w_1 := w_1 - \alpha \frac{dJ(w_1, b)}{dw_1}$$

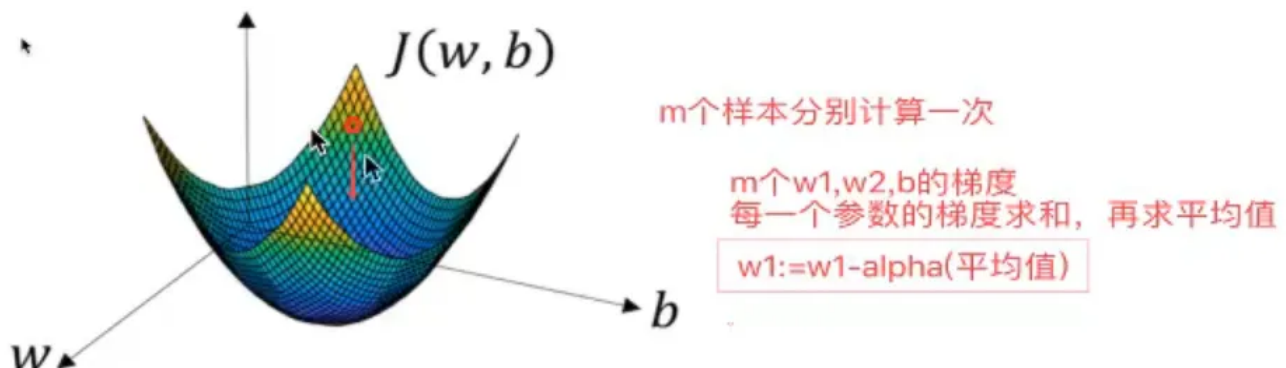
$$w_2 := w_2 - \alpha \frac{dJ(w_2, b)}{dw_2}$$

$$b := b - \alpha \frac{dJ(w, b)}{db}$$

向量化编程

每更新一次梯度时候，在训练期间我们会拥有m个样本，那么这样每个样本提供进去都可以做一个梯度下降计算。所以我们要去做在所有样本上的计算结果、梯度等操作

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(a^{(i)}, y^{(i)})$$



计算参数的梯度为: $d(w_1)^i, d(w_2)^i, d(b)^i$, 这样我们想要得到最终的dw1,dw2,db,如何去设计一个算法计算

伪代码实现:

初始化, 假设

$$J = 0, dw1 = 0, dw2 = 0, db = 0$$

for in m:

$$z^i = w^T x^i + b$$

计算损失

$$a^i = \sigma(z^i)$$

$$J+ = -[y^i \log(a^i) + (1 - y^i) \log(1 - a^i)]$$

每个梯度计算结果相加

$$dz^i = a^i - y^i$$

$$dw1+ = x_1^i dz^i$$

$$dw2+ = x_2^i dz^i$$

$$db+ = dz^i$$

最后求出平均梯度

$$J/ = m$$

$$dw1/ = m$$

$$dw2/ = m$$

w1:=w1-alphadw1 w2:=s2-alphadw2

$$db/ = m$$

向量化优势

由于在进行计算的时候, 最好不要使用for循环去进行计算, 因为有Numpy可以进行更加快速的向量化计

算。在公式 $z = w^T x + b$ 中w,x都可能是多个值, 也就是 $\bar{w} = \begin{pmatrix} w_1 \\ \vdots \\ w_n \end{pmatrix}, x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$

- 导入包

```
1 import numpy as np
2 import time
3 a = np.random.rand(100000)
4 b = np.random.rand(100000)
```

- 第一种方法 (for循环)

```
Python |  
1 # 第一种for循环  
2 c = 0  
3 start = time.time()  
4 for i in range(100000):  
5     c += a[i]*b[i]  
6 end = time.time()  
7 print ("计算所用时间%s" % str(100000*(end-start))+"ms")
```

- 第二种方法 (向量化方式使用np.dot)

```
Python |  
1 # 向量化运算  
2 start =time.time()  
3 c = np.dot(a,b)  
4 end = time.time()  
5 print ("计算所用时间%s" % str(1000*(end-start))+"ms")
```

```
[5]: import numpy as np  
import time
```

```
[7]: a = np.random.rand(100000)  
b = np.random.rand(100000)
```

```
[9]: # 第一种for循环  
c = 0  
start = time.time()  
for i in range(100000):  
    c += a[i]*b[i]  
end = time.time()  
print ("计算所用时间%s" % str(1000*(end-start))+"ms")
```

计算所用时间47.00946807861328ms

.....

```
[43]: # 向量化运算  
start = time.time()  
c = np.dot(a,b)  
end = time.time()  
print ("计算所用时间%s" % str(1000*(end-start))+"ms")
```

计算所用时间6.276607513427734ms

Numpy能够充分的利用并行化，Numpy当中提供了很多函数使用

函数	作用
np.ones or np.zeros	全为1或者0的矩阵
np.exp	指数计算
np.log	对数计算
np.abs	绝对值计算

所以上述的m个样本的梯度更新过程，就是去除掉for循环。原本这样的计算

向量化编程：避免使用for循环进行向量的计算，使用点np.dot等numpy工具，进行操作

向量化实现伪代码

- 思路：

$z^1 = w^T x^1 + b$	$z^2 = w^T x^2 + b$	$z^3 = w^T x^3 + b$
$a^1 = \sigma(z^1)$	$a^2 = \sigma(z^2)$	$a^3 = \sigma(z^3)$

可以变成这样的计算：

$$\bar{w} = \begin{pmatrix} w_1 \\ \vdots \\ w_n \end{pmatrix}, \bar{x} = \begin{pmatrix} \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x^1 & x^2 & x^3 & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

注：w的形状为 (n, 1)，x的形状为 (n, m)，其中n为特征变量，m为样本变量

我们可以让 $Z = W^T X + b = (z^1, z^2, z^3 \cdots z^m) + b = np.dot(W^T, X) + b$ ，得出的结果为 (1, m) 大小

注：大写的W，X为多个样本表示

- 实现多个样本向量化计算的伪代码

初始化,假设n个特征,m个样本

$J = 0, W = np.zeros([n, 1]), b = 0$

$Z = np.dot(W^T, X) + b$

$A = \sigma(Z)$

每个样本梯度计算过程为:

$$dZ = A - Y$$

$$dW = \frac{1}{m} X dZ^T$$

$$db = \frac{1}{m} np.sum(dZ)$$

更新

$$W := W - \alpha dW$$

$$b := b - \alpha db$$

这相当于一次使用了M个样本的所有特征值与目标值，那我们知道如果想多次迭代，使得这个M个样本重复若干次计算

正向传播与反向传播

前面我们所做的整个过程分为两个部分，一个是从前往后的计算出梯度与损失，另外一部分是从后往前计算参数的更新梯度值。所以在神经网络当中会经常出现两个概念，正向传播与反向传播

作业实现：

```
] : def initialize_with_zeros(shape):
    w = np.zeros((shape,1))
    b = 0
    assert(w.shape == (shape,1))
    assert(isinstance(b,float) or isinstance(b,int))
    return w,b
```



```
] : def propagate(w,b,x,y):
    m = X.shape[1]

    # 正向传播
    # 开始
    A= basic_sigmoid(np.dot(w.T, X) + b)
    # 计算损失
    cost = -1 / m * np.sum(Y * np.log(A) + (1-Y) * np.log(1-A))
    # 结束
    dz = A - Y
    dw = 1 / m * np.dot(X, dz, T)
    db = 1 / m * np.sum(dz)
    # 反向传播
    # 开始

    # 结束
    assert(dw.shape == w.shape)
    assert(db.dtype == float)
    cost = np.squeeze(cost)
    assert(cost.shape == ())

    grads = {"dw": dw,
              "db": db}
    return grads,cost
```

```

]: def optionize(w,b,X,Y, num_iteration,learning_rate):
    costs = []
    for i in range(num_iteration):
        grads,cost = propagate(w,b,X,Y)
        dw = grads['dw']
        db = grads['db']
        w = w - learning_rate * dw
        b = b - learning_rate * db

        if i % 100 == 0:
            costs.append(cost)
        if i % 100 == 0:
            print("损失结果 %i: %f" %(i,cost))

    params = {"w":w,
              "b":b}
    grads = {"dw":dw,
             "db":db}
    return params , grads , costs

```

```

]: def model(X_train, Y_train, X_test, Y_test, num_iterations = 2000, learning_rate = 0.5):
    """
    """

    ### 开始

    # 初始化参数
    w, b = initialize_with_zeros(X_train.shape[0])

    # 梯度下降
    # params:更新后的网络参数
    # grads:最后一次梯度
    # costs:每次更新的损失列表
    params, grads, costs = optimize(w, b, X, Y, num_iterations, learning_rate)

    # 获取训练的参数
    # 预测结果
    w = params['w']
    b = params['b']

    Y_prediction_train = predict(w, b, X_train)
    Y_prediction_test = predict(w, b, X_test)
    ### 结束

    # 打印准确率
    print("训练集准确率: {} ".format(100 - np.mean(np.abs(Y_prediction_train - Y_train)) * 100))
    print("测试集准确率: {} ".format(100 - np.mean(np.abs(Y_prediction_test - Y_test)) * 100))

    d = {"costs": costs,
         "Y_prediction_test": Y_prediction_test,
         "Y_prediction_train": Y_prediction_train,
         "w": w,
         "b": b,
         "learning_rate": learning_rate,
         "num_iterations": num_iterations}

    return d

```

在次前提中，需要包含train和test数据集，所以我写的缺少数据来实现，暂时没有结果

这是课程结果：

测试

```
In [24]: d = model(train_x, train_y, test_x, test_y, num_iterations = 2000, learning_rate = 0.005)
```

```
损失结果 0: 0.693147  
损失结果 100: 0.584508  
损失结果 200: 0.466949  
损失结果 300: 0.376007  
损失结果 400: 0.331463  
损失结果 500: 0.303273  
损失结果 600: 0.279880  
损失结果 700: 0.260042  
损失结果 800: 0.242941  
损失结果 900: 0.228004  
损失结果 1000: 0.214820  
损失结果 1100: 0.203078  
损失结果 1200: 0.192544  
损失结果 1300: 0.183033  
损失结果 1400: 0.174399  
损失结果 1500: 0.166521  
损失结果 1600: 0.159305  
损失结果 1700: 0.152667  
损失结果 1800: 0.146542  
损失结果 1900: 0.140872  
训练集准确率: 99.04306220095694  
测试集准确率: 70.0
```

总共50个测试集中，最后得到70% 的正确率，所以正确率还是太低了

